

Паттерны рассуждений и мультиагентная оркестрация enterprise-процессов 1С

Вторая статья цикла NOPiK: почему линейный граф состояний не масштабируется на сложные сценарии, и как декомпозиция на роли (Супервайзер, Аналитик метаданных, Сборщик СКД) с паттернами ReAct и Tree-of-Thoughts решает эту проблему.

Целевая аудитория: Архитекторы решений 1С, ведущие разработчики, специалисты по интеграции ИИ.

Применимость: Внешний оркестратор NOPiK (Python / LangGraph); контур 1С:Предприятие 8.3 не меняется относительно статьи 1.

Серия: NOPiK - автономный ИИ-агент для 1С:Предприятие 8 (статья 2 из 7).

Оглавление:

1. От линейного графа к ролям
2. Три роли NOPiK
3. Паттерн ReAct у Аналитика метаданных
4. Паттерн Tree-of-Thoughts у Сборщика СКД
5. Сборка графа: супервайзер и conditional edges
6. Тестирование маршрутизации
7. Что дальше

Диагностика: от линейного графа к ролям

В статье 1 цикла граф состояний NOPiK был намеренно упрощен до линейной цепочки `analyze_intent` → `plan_executor` → `deliver_to_1c`. Такая цепочка наглядно демонстрирует переход от одного гигантского промпта к формальному FSM, но у нее есть системный недостаток: один узел `plan_executor` отвечает сразу за все прикладные сценарии - и построение отчета, и диагностику ошибки, и общий ответ. По мере роста числа сценариев этот узел неизбежно превращается в новую версию того самого «монолитного промпта», от которого мы уходили.

Решение - декомпозиция по ролям с явным супервайзером, а не наращивание ветвлений внутри одного узла: та же логика, что в микросервисной архитектуре - каждая роль отвечает за одну зону ответственности и не обязана знать о существовании соседних ролей.

Структура: три роли NOPiK

Начиная с этой статьи оркестратор NOPiK построен вокруг трех ролей:

- **Супервайзер (Supervisor)** - единственный узел, которому разрешено видеть весь маршрут целиком. Классифицирует намерение пользователя и решает, какая роль обработает запрос дальше. Не выполняет прикладной работы сам.
- **Аналитик метаданных (Metadata Analyst)** - находит объекты метаданных 1С (справочники, регистры, реквизиты), релевантные запросу, работая по паттерну **ReAct**.
- **Сборщик СКД (SKD Collector)** - строит и «исполняет» отчет через Систему компоновки данных, работая по паттерну **Tree-of-Thoughts**.

Такое разделение прямо соответствует терминологии HSM из статьи 1: три роли образуют суперсостояние `WORKING`, а супервайзер реализует функцию переходов `δ` этого суперсостояния.

Паттерн ReAct: Аналитик метаданных

ReAct (Reasoning + Acting) - паттерн, в котором рассуждение модели (Thought) перемежается с вызовом инструмента (Action) и анализом его результата (Observation), и цикл повторяется, пока не будет собрано достаточно информации.

Наивная реализация зашивает цикл `while` внутрь одной функции на Python - и тогда граф состояний перестает быть графом состояний, превращаясь обратно в скрытый монолитный скрипт с ИИ внутри. В `NOPIk` цикл ReAct вынесен на уровень графа: каждый вызов узла `metadata_analyst_node` выполняет **ровно один** шаг Thought → Action → Observation, а решение «повторить, эскалировать или закончить» принимает `conditional edge` графа - `route_from_metadata_analyst`.

Python - graph/agents/metadata_analyst.py

UTF-8

[Открыть файл](#)

```

1 def metadata_analyst_node(state: AgentState) -> dict:
2     iteration = state["react_iteration"] + 1
3     thought = f"Нужно найти объекты метаданных, релевантные запросы: '{state['user_query']}'"
4     found = discover_metadata_tool(state["user_query"])
5
6     if found:
7         observation = f"Найдено {len(found)} объектов: {' '.join(found)}."
8     else:
9         observation = "Наблюдение неполно: по запросу не найдено ни одного объекта метаданных"
10
11     step = {"thought": thought, "action": "discover_metadata_tool(query)", "observation": observation}
12
13     return {
14         "react_steps": state["react_steps"] + [step],
15         "react_iteration": iteration,
16         "discovered_metadata": state["discovered_metadata"] + found,
17     }

```

Guard, ограничивающий число итераций (`MAX_REACT_STEPS = 3`), - это тот самый Global Guard из теории HSM: лимит вешается один раз на уровне суперсостояния и защищает от бесконечного цикла «модель не нашла ответ - модель пробует снова с тем же результатом».

Python - graph/agents/supervisor.py

UTF-8

[Открыть файл](#)

```

1 def route_from_metadata_analyst(state: AgentState) -> str:
2     last_step = state["react_steps"][-1] if state["react_steps"] else None
3     need_more_info = last_step is not None and "неполно" in last_step["observation"]
4
5     if need_more_info and state["react_iteration"] < MAX_REACT_STEPS:
6         return "metadata_analyst"
7     if state["role"] == "SKD_COLLECTOR":
8         return "skd_collector"
9     return "deliver_to_1c"

```

В боевой системе `discover_metadata_tool` обращается к MCP-серверу 1C за реальным списком объектов метаданных (протокол MCP - тема статьи 3 этого цикла); в текущей версии репозитория это детерминированный мок, чтобы граф переходов оставался полностью воспроизводимым в модульных тестах.

Паттерн Tree-of-Thoughts: Сборщик СКД

Tree-of-Thoughts (ToT) - паттерн, в котором вместо последовательного уточнения одной гипотезы (как в ReAct) модель за один шаг порождает несколько альтернативных планов

решения задачи, оценивает каждый и исполняет только план-победитель. Для построения отчета СКД это особенно оправдано: неверно угаданный макет компоновки данных нельзя «доуточнить» маленькими шагами - его нужно сравнивать с альтернативами целиком.

Python — graph/agents/skd_collector.py (генерация и оценка планов)

UTF-8

[Открыть файл](#)

```
1 def generate_candidate_plans(query: str, discovered_metadata: list[str]) -> list[dict]:
2     base_objects = discovered_metadata or ["РегистрНакопления.ТоварыНаСкладах"]
3
4     candidates = [
5         {
6             "description": f"Простой отчет по остаткам: группировка по номенклатуре",
7             "score": 0.6,
8         },
9         {
10            "description": f"Детализированный отчет: группировка по складу и номенклатуре",
11            "score": 0.9 if len(base_objects) > 1 else 0.7,
12        },
13        {
14            "description": "Сводный отчет без детализации (только итоговая сумма).",
15            "score": 0.3,
16        },
17    ]
18    return sorted(candidates, key=lambda c: c["score"], reverse=True)
```

Если запрос перед этим прошел через Аналитика метаданных и тот нашел несколько объектов, оценка «детализированного» плана растет автоматически (0.9 против 0.7) - число найденных объектов метаданных напрямую влияет на выбор ветки дерева гипотез. Узел `skd_collector_node` берет план с максимальной оценкой и передает его в `execute_skd_report_tool` (в боевой системе - реальный вызов СКД через MCP-сервер).



Сборка графа: супервайзер и conditional edges

LangGraph выражает ветвление не через `if` внутри узла, а через `add_conditional_edges`: узел меняет состояние, а отдельная чистая функция-маршрутизатор его читает и решает, куда идти дальше. Это разделение - ключевое отличие управляемого графа состояний от произвольного агентного скрипта.

Python - graph/build.py

UTF-8

[Открыть файл](#)

```
1 workflow.add_edge(START, "supervisor")
2
3 workflow.add_conditional_edges(
4     "supervisor",
```

```

5     route_from_supervisor,
6     {
7         "metadata_analyst": "metadata_analyst",
8         "skd_collector": "skd_collector",
9         "general_answer": "general_answer",
10    },
11 )
12
13 workflow.add_conditional_edges(
14     "metadata_analyst",
15     route_from_metadata_analyst,
16     {
17         "metadata_analyst": "metadata_analyst",
18         "skd_collector": "skd_collector",
19         "deliver_to_1c": "deliver_to_1c",
20     },
21 )
22
23 workflow.add_edge("skd_collector", "deliver_to_1c")
24 workflow.add_edge("general_answer", "deliver_to_1c")
25 workflow.add_edge("deliver_to_1c", END)

```

Обратите внимание: узел `skd_collector` достигим двумя путями - напрямую от супервайзера (если намерение сразу однозначно про отчет) и через `metadata_analyst` (если сначала потребовалось найти объекты метаданных). Обе ветки сходятся в единственном терминальном узле `deliver_to_1c`, что сохраняет свойство FSM из статьи 1: терминальные состояния ($F \subseteq S$) должны быть предсказуемым и небольшим множеством, а не размазаны по графу.

Тестирование: маршрутизация графа

Поскольку узлы и маршрутизаторы - обычные чистые функции Python, весь граф проверяется модульными тестами `pytest` без обращения к реальному LLM или MCP-серверу: достаточно собрать `AgentState` и провалидировать, в какую роль ушел запрос и сколько шагов ReAct потребовалось.

Python - tests/test_metadata_analyst.py

UTF-8

[Открыть файл](#)

```

1 def test_metadata_analyst_route_stops_after_max_react_steps_on_unknown_query():
2     app = build_agent_app()
3     config = {"configurable": {"thread_id": "conv-test-3"}}
4     result = app.invoke(_base_state("покажи метаданные для чего-то незнакомого"), со
5
6     assert result["role"] == "METADATA_ANALYST"

```

```
7     assert len(result["react_steps"]) == MAX_REACT_STEPS
8     assert result["discovered_metadata"] == []
```

Такой тест фиксирует именно то поведение, которое обещает Guard из теории HSM: агент не зависает в цикле ReAct навсегда, а после трех безуспешных попыток детерминированно передает управление дальше по графу.

👉 Итоги: что дальше

Оба инструмента этой статьи - `discover_metadata_tool` и `execute_skd_report_tool` - сегодня являются моками. В статье 3 цикла они станут настоящими вызовами к MCP-серверу 1C: разберем протокол Model Context Protocol, валидацию ответов через Instructor/Pydantic, борьбу с нечувствительностью LLM к регистру имен объектов метаданных и генерацию кликабельных навигационных ссылок вида `e1cib/data/...` прямо в ответе агента.

Полный код ролей - в `NOPIk/orchestrator/graph/agents/` репозитория проекта.

Как в вашей практике решается баланс между глубиной цикла ReAct (сколько итераций уточнения допустимо) и стоимостью лишних вызовов LLM? Делитесь подходами в комментариях - обсудим в следующих статьях цикла.

Теги: LangGraph, Мультиагентные системы, ReAct, Tree-of-Thoughts, Граф состояний, Супервайзер, Оркестрация агентов, СКД, MCP, LLM, Enterprise AI, 1C:Предприятие 8.3, Python, NOPIk, Автоматизация 1C, Иерархический автомат, HSM, FSM